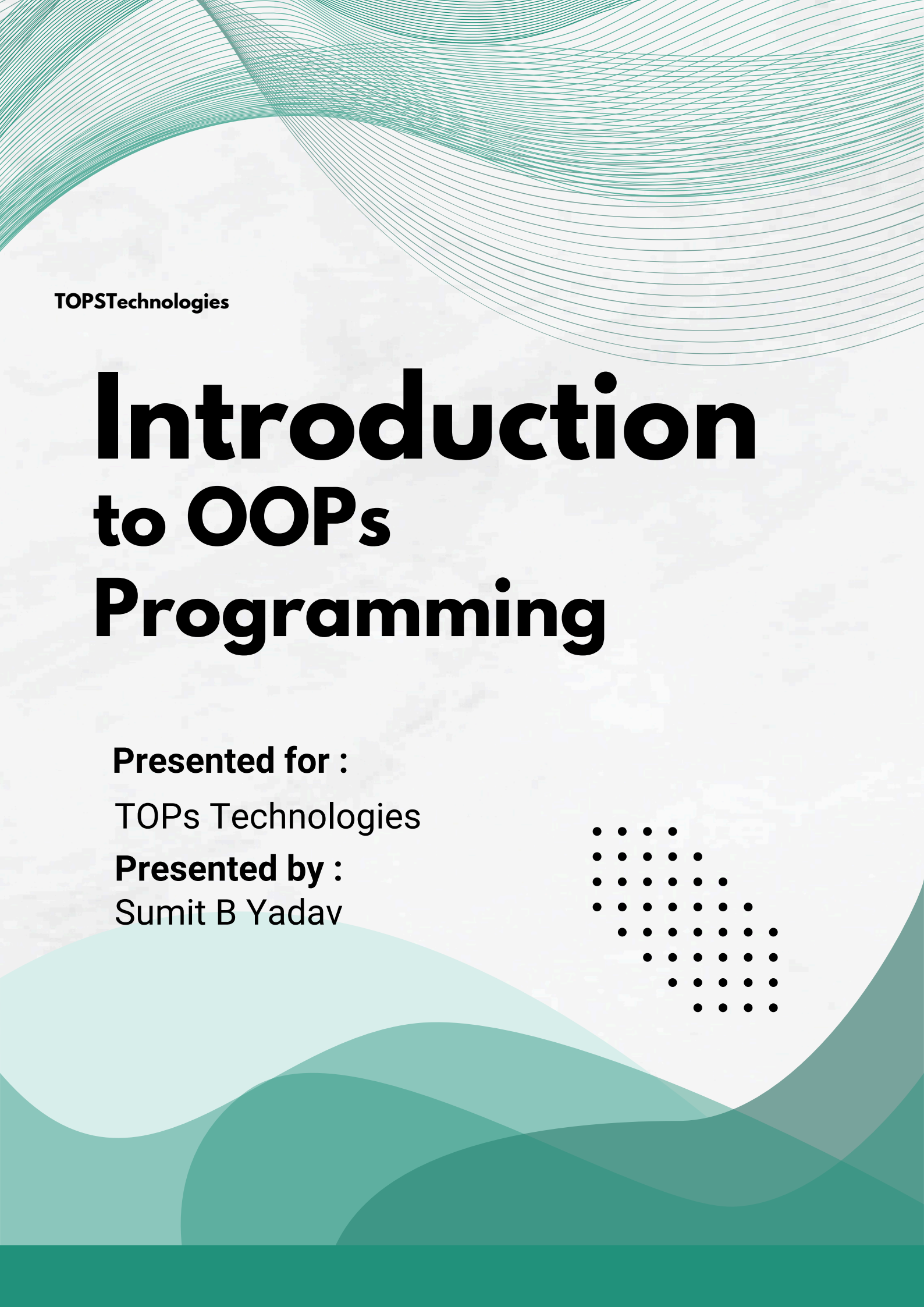




TOPSTechnologies

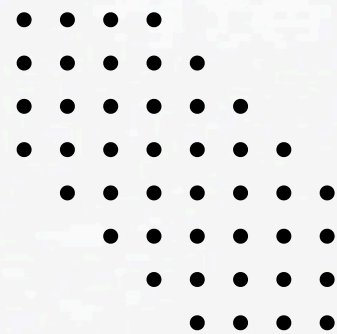
Introduction to OOPs Programming

Presented for :

TOPs Technologies

Presented by :

Sumit B Yadav



Basic Structure of a C++ Program

Que 1

Conditional Statements in C++

Conditional statements in C++ are used to make decisions based on conditions or logical expressions. These statements allow the program to take different actions depending on whether certain conditions evaluate to true or false. The main conditional statements in C++ are:

- if statement
- if-else statement
- if-else if-else ladder
- switch statement

These statements enable the program to control the flow of execution by testing conditions and deciding which block of code to execute.

1. if Statement

The if statement is used to execute a block of code if a specified condition evaluates to true. If the condition is false, the block of code is skipped.

if-else Statement

The if-else statement allows you to specify two blocks of code: one that runs if the condition is true and another that runs if the condition is false.

Basic Structure of a C++ Program

if-else if-else Ladder

The if-else if-else ladder allows multiple conditions to be checked sequentially. If the first condition is false, the program checks the next condition, and so on. If none of the conditions are true, the final else block is executed.

Switch Statement

The switch statement is used to select one of many code blocks to be executed based on the value of an expression. Unlike multiple if-else conditions, a switch statement can be more efficient and readable when checking a single variable against multiple possible values.

Que 2

For Loop

A for loop is typically used when the number of iterations is known in advance. It allows you to initialize a variable, check a condition, and modify the loop variable all in one statement.

While Loop

A while loop is used when the number of iterations is not necessarily known ahead of time. The loop continues as long as the specified condition evaluates to true.

Basic Structure of a C++ Program

Do-while Loop

A do-while loop is similar to the while loop, but with one key difference: the condition is checked after the loop body is executed. This guarantees that the loop will run at least once, even if the condition is false.

Que 3

The break Statement

The break statement is used to immediately exit a loop, regardless of whether the loop's condition has been fully evaluated. When a break statement is encountered inside a loop, the loop terminates and the program continues with the next statement after the loop.

Use Case: Exiting a loop early when a certain condition is met (e.g., when a search is successful).

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i == 5) {
            break; // Exit the loop when i reaches 5
        }
        cout << i << " ";
    }
    cout << "\nLoop terminated early." << endl;

    return 0;
}
```

Basic Structure of a C++ Program

The continue Statement

The continue statement is used to skip the current iteration of the loop and proceed to the next iteration. When continue is encountered, the remaining code in the current iteration is skipped, and the loop proceeds to the next iteration (re-evaluating the loop condition).

Use Case: Skipping specific iterations of the loop when a certain condition is met (e.g., skipping even numbers in a loop).

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i % 2 == 0) {
            continue; // Skip the even numbers
        }
        cout << i << " ";
    }
    cout << "\nLoop finished." << endl;

    return 0;
}
```

Basic Structure of a C++ Program

The break Statement

The *break* statement is used to immediately exit a loop, regardless of whether the loop's condition has been fully evaluated. When a *break* statement is encountered inside a loop, the loop terminates and the program continues with the next statement after the loop.

Use Case: Exiting a loop early when a certain condition is met (e.g., when a search is successful).

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; ++i) {
        if (i == 5) {
            break; // Exit the loop when i reaches 5
        }
        cout << i << " ";
    }
    cout << "\nLoop terminated early." << endl;

    return 0;
}
```


Basic Structure of a C++ Program

Nested Control Structures

A nested control structure refers to placing one control statement (like an if, for, while, or switch statement) inside another control structure. These are useful when you need to check or loop through multiple conditions or groups of conditions.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int age = 20;
    int score = 85;

    if (age >= 18) {
        // Outer if: check if the person is an adult
        if (score >= 50) {
            // Inner if: check if the person passed the exam
            cout << "You are an adult and have passed the exam." << endl;
        } else {
            cout << "You are an adult but have not passed the exam." << endl;
        }
    } else {
        cout << "You are not an adult." << endl;
    }

    return 0;
}
```